

assignment5

DSA Assignment #5: 20260401 cs201 Mock Exam

Updated 2026-04-02 15:20 GMT+8

Compiled by 徐前 物理学院 (2026 Spring)

说明:

1. 解题与记录:

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge，Codeforces，LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **提交安排:** 提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交:** 如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

E02039: 反反复复

matrix, <http://cs101.openjudge.cn/practice/02039/>

思路:

用 i 枚举列号，然后输出字符依次位于原字符串的 $i, 2m - 1 - i, 2m + i, 4m - 1 - i, 4m + i, \dots$ 的位置，依次输出即可

代码:

```
m = int(input())
s = input()
for i in range(m):
    j = i
    k = 2 * m
    while j < len(s):
```

```
print(s[j], end = '')
j = k - 1 - j
k += 2 * m
```

代码运行截图 (至少包含有"Accepted")

#52182081提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
m = int(input())
s = input()
for i in range(1, m + 1):
    j = i
    k = 2 * m
    while j <= len(s):
        print(s[j - 1], end = '')
        j = k + 1 - j
        k += 2 * m
```

基本信息

#: 52182081
题目: 02039
提交人: 25n2500011619
内存: 3624kB
时间: 26ms
语言: Python3
提交时间: 2026-04-02 14:19:39

E02092: Grandpa is Famous

implementation, <http://cs101.openjudge.cn/practice/02092/>

思路:

统计次数count, 然后输出次数count排第二的所有value

代码:

```
while True:
    n, m = map(int, input().split())
    if n == 0:
        break
    cnt = [0] * 10001
    for i in range(n):
        a = list(map(int, input().split()))
        for j in a:
            cnt[j] += 1
    mx, sec = 0, 0
    for x in cnt:
        if x > mx:
            sec, mx = mx, x
        elif x > sec:
            sec = x
    for i, x in enumerate(cnt):
        if x == sec:
```

```
print(i, end = ' ')\nprint('')
```

代码运行截图 (至少包含有"Accepted")

#52182151提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
while True:\n    n, m = map(int, input().split())\n    if n == 0:\n        break\n    cnt = [0] * 10001\n    for i in range(n):\n        a = list(map(int, input().split()))\n        for j in a:\n            cnt[j] += 1\n    mx, sec = 0, 0\n    for x in cnt:\n        if x > mx:\n            sec, mx = mx, x\n        elif x > sec:\n            sec = x\n    for i, x in enumerate(cnt):\n        if x == sec:\n            print(i, end = ' ')\n    print('')
```

基本信息

#: 52182151
题目: 02092
提交人: 25n2500011619
内存: 4480kB
时间: 163ms
语言: Python3
提交时间: 2026-04-02 14:24:32

M02774: 木材加工

binary search, <http://cs101.openjudge.cn/practice/02774/>

思路:

二分可以切的长度, 然后注意特判输出为0的情况, 否则会RE。

代码:

```
n, k = map(int, input().split())\na = [int(input()) for _ in range(n)]\nif sum(a) < k:\n    print(0)\nelse:\n    l, r = 1, sum(a) // k\n    while l <= r:\n        mid = (l + r) >> 1\n        if sum(y // mid for y in a) >= k:\n            l = mid + 1\n        else:\n            r = mid - 1\n    print(r)
```

代码运行截图 (至少包含有"Accepted")

#52182183提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
n, k = map(int, input().split())
a = [int(input()) for _ in range(n)]
if sum(a) < k:
    print(0)
else:
    l, r = 1, sum(a) // k
    while l <= r:
        mid = (l + r) >> 1
        if sum(y // mid for y in a) >= k:
            l = mid + 1
        else:
            r = mid - 1
    print(r)
```

基本信息

#: 52182183
题目: 02774
提交人: 25n2500011619
内存: 3972kB
时间: 44ms
语言: Python3
提交时间: 2026-04-02 14:26:59

M04077: 出栈序列统计

dp, dfs, math, <http://cs101.openjudge.cn/practice/04077/>

思路:

考场上忘记了卡特兰数的公式, 所以用DP做。

$dp[i][j]$ 表示未入栈和栈中的数, 转移方程 $dp[i][j] = dp[i-1][j+1] + dp[i][j-1]$, 初始化 $dp[0][j] = 1$ 。

代码:

```
## 考场版
n = int(input())
dp = [[0 for j in range(n - i + 1)] for i in range(n + 1)]
for j in range(n + 1):
    dp[0][j] = 1
for i in range(1, n + 1):
    dp[i][0] = dp[i-1][1]
    for j in range(1, n - i + 1):
        dp[i][j] = dp[i-1][j+1] + dp[i][j-1]
print(dp[n][0])

## AI 优化版
n = int(input())
dp = [1] * (n + 1)

for i in range(1, n + 1):
    dp[0] = dp[1]
    for j in range(1, n - i + 1):
        dp[j] = dp[j+1] + dp[j-1]
```

```
print(dp[0])

## 卡特兰数
from math import comb

n = int(input())
print(comb(2 * n, n) // (n + 1))
```

代码运行截图 (至少包含有"Accepted")

#52182218提交状态

查看 提交 统计 提问

状态: Accepted

源代码

```
n = int(input())
dp = [[0 for j in range(n - i + 1)] for i in range(n + 1)]
for j in range(n + 1):
    dp[0][j] = 1
for i in range(1, n + 1):
    dp[i][0] = dp[i - 1][1]
    for j in range(1, n - i + 1):
        dp[i][j] = dp[i - 1][j + 1] + dp[i][j - 1]
print(dp[n][0])
```

基本信息

#: 52182218
 题目: 04077
 提交人: 25n2500011619
 内存: 3640kB
 时间: 28ms
 语言: Python3
 提交时间: 2026-04-02 14:30:18

M30637: 合法出栈序列pub

stack, <http://cs101.openjudge.cn/practice/M30637/>

思路:

建立每个字符到位置的映射, 模拟进出栈过程, 如果新的出栈数pos大于目前的最大数, 则一直push直到pos位于栈中, 然后再pop, 如果空栈或 pos!=stack.pop() 则出栈序列非法。

代码

```
s0: str = input()
dic = {x: i for i, x in enumerate(s0)}
n = len(s0)
while True:
    try:
        p = input()
        if len(p) != n:
            print('NO')
            continue
        mx = 0
        ans = True
        stack = []
        for c in p:
            if c not in dic:
```

```

        ans = False
        break
    pos = dic[c]
    while mx <= pos:
        stack.append(mx)
        mx += 1
    if not stack or pos != stack.pop():
        ans = False
        break
    print('YES' if ans else 'NO')
except EOFError:
    break

```

(至少包含有"Accepted")

状态: Accepted

源代码

```

s0 = input()
dic = {x: i for i, x in enumerate(s0)}
n = len(s0)
while True:
    try:
        p = input()
        if len(p) != n:
            print('NO')
            continue
        mx = 0
        ans = True
        stack = []
        for c in p:
            if c not in dic:
                ans = False
                break
            pos = dic[c]
            while mx <= pos:
                stack.append(mx)
                mx += 1
            if stack and pos == stack.pop():
                continue
            else:
                ans = False
                break
        print('YES' if ans else 'NO')
    except EOFError:

```

基本信息

#: 52182717
 题目: M30637
 提交人: 25n2500011619
 内存: 3656kB
 时间: 25ms
 语言: Python3
 提交时间: 2026-04-02 14:58:44

T30102:完美交易窗口

monotonic stack, <http://cs101.openjudge.cn/practice/T30102/>

思路:

本题要找最长区间 $[i..j]$, 满足:

- (h_i) 是区间最小值 (且内部不能再出现 (h_i))
- (h_j) 是区间最大值 (且内部不能再出现 (h_j))

- $(h_j > h_i)$

用单调栈维护候选，`pos` 记录当前 `j` 可配对的最早买入点。

代码

```
import sys

def solve() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return

    inf = 2**31
    n = data[0]
    arr = [inf] + data[1 : n + 1]

    ans = 0
    st = []

    st_append = st.append
    st_pop = st.pop

    for j in range(n + 1):
        aj = arr[j]
        mn = inf
        pos = -1

        while st and st[-1][1] < aj:
            tmp = st_pop()[2]
            if tmp != -1:
                at = arr[tmp]
                if at < mn:
                    mn = at
                    pos = tmp

        if st:
            top_best = st[-1][2]
            if top_best != -1:
                at = arr[top_best]
                if at < mn:
                    mn = at
                    pos = top_best
            else:
                st[-1][2] = pos
        else:
            st[-1][2] = pos
```

```

        st_append([j, aj, j])

    if pos != -1 and arr[pos] < aj:
        length = j - pos + 1
        if length > ans:
            ans = length

sys.stdout.write(str(ans))

if __name__ == '__main__':
    solve()

```

源代码的常数过大问题

1. **I/O 很慢**： `int(input())` 读 100 万行，调用次数巨大。
2. **栈元素是嵌套 list**： `st[-1][2]`、`st.pop()[2]` 这类多层索引和对象分配开销大。
3. **循环里大量 Python 级操作**： `while` + 多次比较 + 多次索引，都在解释器层执行。
4. **`float('inf')` 与 `int` 混用**： 比较可行但有额外类型处理成本。
5. **频繁调用 `max`、重复 `a[...]` 取值**： 小开销累计后很明显。

(至少包含有"Accepted")

#52206285提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: **Accepted**

源代码

```

import sys

def solve() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return

    inf = 2 ** 31
    n = data[0]
    arr = [inf] + data[1 : n + 1]

    ans = 0

    # --- 核心优化: 预分配并行数组替代嵌套 List ---
    # 最大可能有 n+1 个元素入栈, 直接分配好内存, 避免 list 动态扩容
    # 栈内元素结构: 数值 value, 当前管辖区间内的最佳左端点下标 best_pos
    st_val = [0] * (n + 1)

```

基本信息

#: 52206285
 题目: T30102
 提交人: 25n2500011619
 内存: 67584kB
 时间: 495ms
 语言: Python3
 提交时间: 2026-04-05 11:26:21

2. 学习总结和个人收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2026spring每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。

AC5，考场上的做题感觉和自己练习的确有很大差别，一是时间紧张，二是会有最优的方法临时想不起来，只能用暴力方法去做。比如T4.出栈序列统计就忘记了卡特兰数的公式，就改成用DP去做。另一个感受是考试题目需要注意细节的判断，比如T3 需要特判0的情况，T5需要判断字符不在原串中或长度不匹配的情况，这些不注意会影响做题进度。

T6 完美交易窗口是一个比较综合的单调栈题目，本题不仅需要维护第一个大于等于或小于等于 h_j 的数的位置，还需要动态维护一个区间内的最大最小值。数算群中有讨论了 $O(n\log n)$ 的单调栈+二分分解法，其实可以用贪心求解。

令 $L[j]$ 表示在 j 左侧第一个 $\geq h_j$ 的位置（如果没有则为 -1）。那么 i 必须满足 $i > L[j]$ 。对于任意固定的右端点 j ，我们希望找到一个**最小的 i** ，满足 $L[j] < i < j$ ，且 i 右边没有小于等于 i 的数。那么实际上，我们只要在限定域 $(L[j], j)$ 内找到**最右边的绝对最小值位置 pos** ，它一定是最优的左端点，因为任何 $x < pos$ 处的数一定 $h[x] > h[pos]$ ，而任意 pos 右边的数都不如 pos 优。

我们可以使用状态合并来维护每个位置所管辖的最小值的下标 pos ，当栈中元素弹出时，其所管辖的最小值的下标（ pos ）和值本身，可以向左递推合并给新的栈顶。这样可以规避线段树等数据结构，将时间复杂度降到了严格的 $O(N)$ 。

代码

```
import sys

def solve() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return

    inf = 2 ** 31
    n = data[0]
    arr = [inf] + data[1 : n + 1]

    ans = 0

    # 栈内元素结构：数值 value，当前管辖区间内的最佳左端点下标 best_pos
    st_val = [0] * (n + 1)
    st_pos = [0] * (n + 1)
    top = -1 # 栈顶指针，-1 表示空栈

    for j in range(n + 1):
        aj = arr[j]
        mn = inf
        pos = -1

        # 1. 弹出比当前 aj 小的元素，合并它们所管辖的最优左端点
        while top >= 0 and st_val[top] < aj:
```

```

    tmp_pos = st_pos[top]
    top -= 1 # 手动弹栈
    at = arr[tmp_pos]
    # 严格小于才更新，保证在有相同最小值时，保留“最右边的最小值”
    if at < mn:
        mn = at
        pos = tmp_pos

# 2. 将弹出的最优信息汇总到当前栈顶（即左侧第一个 >= aj 的元素）
if top >= 0:
    top_best = st_pos[top]
    at = arr[top_best]
    if at < mn:
        # 栈顶自带的最优值更小，提取出来用作计算
        mn = at
        pos = top_best
    else:
        # 弹出的最优值更小或相等，将其合并到栈顶的管辖域中
        st_pos[top] = pos

# 3. 压栈：直接通过指针操作
# 把自己压入栈中，初始管辖域最优左端点就是自己
top += 1
st_val[top] = aj
st_pos[top] = j

# 4. 如果找到了有效的、且满足 h_i < h_j 的最小端点，计算区间长度
if pos != -1 and mn < aj:
    length = j - pos + 1
    if length > ans:
        ans = length

sys.stdout.write(str(ans))

if __name__ == '__main__':
    solve()

```

AI 建议

1. **零对象分配**：整个 `for` 循环内部没有任何对象的创建与销毁（没有 `[]` 括号，也没有解包）。
2. **零函数调用开销**：去掉了 `st.append()` 和 `st.pop()` 的方法调用开销，取而代之的是纯粹的整数加减 `top += 1` 和 `top -= 1`，这在解释型语言里提速极为显著。

3. **缓存友好性提升**：一维数组 `st_val` 和 `st_pos` 在底层是通过 C 数组（存着对 `int` 的指针）实现的，相比起指针指向不同的 `list` 对象，寻址跳跃更少。